

CSE 539 Applied Cryptography — Project Report TLS Handshake with PAKE Name: Diveet Kaushik Due: May 8, 2026

1. System Architecture 1.1 Overview

This project implements a simplified TLS 1.3-style handshake between a client and a server, augmented with a Password Authenticated Key Exchange (PAKE) protocol based on OPAQUE. The system achieves mutual authentication: the server is authenticated via a PKI certificate and the client is authenticated via its password, without ever transmitting the password over the network.

1.2 Parties

Certificate Authority (CA): Pre-generates an ECDSA P-256 key pair and issues a self-signed certificate. The CA public key is distributed to clients as a trust anchor. The CA is emulated as static pre-generated files (ca/ca_cert.pem, ca/ca_key.pem).

Server: Runs as a separate process listening on 127.0.0.1:8443. Holds an ECDSA key pair signed by the CA, an OPRF key per registered user, and a stored encrypted envelope per user.

Client: Runs as a separate process. Knows only the CA public key, a username, and a password. Derives all other keying material during the protocol.

1.3 Components

Module File Responsibility
 PKI & Certificates ca/ca_keygen.py Key generation, certificate signing
 Digital Signature crypto/signature.py ECDSA sign, verify, load keys/certs
 OPRF crypto/oprf.py Oblivious PRF for password blinding
 Key Derivation crypto/kdf.py HKDF extract + expand
 Authenticated Encryption crypto/aes_gcm.py AES-GCM encrypt/decrypt
 Server server/server.py Registration, handshake, secure channel
 Client client/client.py Registration, handshake, secure channel
 User Database db/users.json Stores OPRF key + encrypted envelope per user

1.4 Protocol Flow Diagram

```

text CLIENT SERVER || |--- ClientHello (username, alpha=H(pwd)^r) --->|| | [OPRF evaluate: beta =
alpha^k] |<-- ServerCert, beta, EphPub_S, Signature -----| || | [Verify CA signature on cert] || [Verify
server ECDSA signature] || [Recover rw = beta^(1/r) = H(pwd)^k] || [Decrypt envelope -> client keypair] |
| [Generate ephemeral key EphPub_C] || | |--- EphPub_C ----->|| | | [Both
compute: shared = ECDH(EphPriv, EphPub)] | [Both compute: keys = HKDF(shared, transcript)] || |<--
MAC_server (key confirmation) -----| |--- MAC_client (key confirmation) ----->|| | |====
AES-GCM Encrypted Channel =====|
  
```

2. Security Intuition 2.1 Secrecy

All post-handshake messages are encrypted using AES-GCM with a 256-bit application traffic key derived via HKDF. AES-GCM provides IND-CCA2 security — an attacker observing ciphertexts cannot learn any information about the plaintexts. A fresh 96-bit random nonce is used per message, preventing IV reuse attacks.

2.2 Integrity

AES-GCM combines encryption with a Galois/Counter Mode authentication tag. Any tampering with a ciphertext — even a single bit flip — causes the tag verification to fail and the message to be rejected. This defends against chosen-ciphertext attacks including padding oracle attacks that affected earlier protocols such as SSL 3.0 with MAC-then-Encrypt.

2.3 Server Authenticity

The server's identity is bound to its X.509 certificate, signed by the CA using ECDSA over P-256. During the handshake, the server signs its ephemeral public key and OPRF response with its long-term private key. The client verifies:

The CA's signature on the server certificate.

The server's signature over the handshake transcript.

This prevents man-in-the-middle attacks where an attacker impersonates the server.

2.4 Client Authenticity (OPAQUE / PAKE)

The client is authenticated via the OPAQUE protocol, which integrates an Oblivious PRF (OPRF) with key exchange. During registration, the client uses OPRF to derive a password-hardened key $rw = H(pwd)^k$ without the server ever learning the password or the client ever learning the OPRF key k . The client encrypts its long-term keypair under rw to produce an envelope, stored at the server.

During authentication, the client re-runs OPRF to recover rw , decrypts the envelope to recover its keypair, and participates in the key exchange. An adversary without the correct password cannot decrypt the envelope, and the derived session key therefore depends on the password — ensuring only the legitimate user authenticates successfully.

2.5 Forward Secrecy

Both parties generate fresh ephemeral ECDH key pairs for every session. The session key is derived from these ephemeral keys via HKDF. Compromise of the server's long-term private key after the fact does not reveal past session keys, since those keys depended on ephemeral values that are discarded after each session.

2.6 Key Confirmation

After key derivation, both parties exchange HMAC-based key confirmation messages:

Server sends $MAC(K_{hs_server}, "server\ finished" \parallel transcript_hash)$

Client sends $MAC(K_{hs_client}, "client\ finished" \parallel transcript_hash)$

This ensures both parties derived the same session key from the same transcript, and detects any active tampering with handshake messages.

3. Implementation 3.1 PKI and Certificate Generation (ca/ca_keygen.py)

The CA generates an ECDSA P-256 private key and issues a self-signed X.509 certificate using Python's cryptography library. The server generates its own key pair and obtains a certificate signed by the CA. These are stored as PEM files and distributed statically, emulating a real PKI without requiring a live CA process.

3.2 OPRF (crypto/oprf.py)

The OPRF is implemented over the NIST P-256 elliptic curve. The client hashes the password to a curve point $H(pwd)$ using a try-and-increment method, then blinds it by scalar multiplication with a random r .

The server evaluates $\beta = \alpha^k$ using ECDH. The client finalizes to recover $rw = \text{HMAC}(\text{pwd}, \beta)$, a password-hardened key that neither party can compute alone.

3.3 Key Derivation (crypto/kdf.py)

HKDF is implemented in two phases following RFC 5869:

Extract: $\text{PRK} = \text{HMAC}(\text{salt}, \text{IKM})$ — converts the raw ECDH shared secret into a uniformly random key.

Expand: Multiple keys derived via $\text{HKDF-Expand}(\text{PRK}, \text{context_label})$ — producing separate handshake traffic keys and application traffic keys for client and server, bound to the session transcript hash.

3.4 Authenticated Encryption (crypto/aes_gcm.py)

AES-256-GCM is used for all post-handshake communication. Each message prepends a fresh 96-bit random nonce to the ciphertext. The GCM authentication tag provides integrity verification. This implements the Encrypt-then-MAC paradigm in an integrated scheme, which is the most secure composition.

3.5 Digital Signatures (crypto/signature.py)

Server messages during the handshake are signed using ECDSA over P-256 with SHA-256. The client verifies the CA's signature on the server certificate using the CA's pre-distributed public key, then verifies the server's signature over the handshake transcript.

3.6 Server (server/server.py)

The server handles two phases:

Registration: Receives the blinded password from the client, evaluates OPRF, and stores the OPRF key and encrypted envelope.

Handshake: Sends certificate and OPRF response signed with its private key, receives the client ephemeral public key, derives session keys via HKDF, performs key confirmation, and establishes an AES-GCM encrypted channel.

3.7 Client (client/client.py)

The client handles two phases:

Registration: Blinds the password, sends to server, recovers rw from the OPRF response, encrypts its long-term keypair under rw , and sends the envelope to the server.

Handshake: Verifies the server certificate and signature, recovers rw via OPRF, decrypts the envelope to recover its keypair, completes the ephemeral ECDH exchange, derives session keys, confirms keys, and communicates over AES-GCM.